DESENVOLVIMENTO DE SISTEMAS • BACK-END • LARAVEL 12

APIs: JSON, XML

testando tudo com o Insomnia 🧪

O que é uma API, como os sistemas trocam dados em JSON e XML e como construir e testar a API de Equipamentos no nosso projeto Laravel.

O que é uma API?

API (Application Programming Interface) é a ponte que deixa dois sistemas conversarem.



Cliente

Faz o pedido

App, site, front-end



Garçom = API

Leva o pedido e traz o prato

Laravel (rotas + controller)



Cozinha

Prepara o que foi pedido

Servidor + banco de dados

💡 O cliente não precisa saber **como** a cozinha funciona — só como fazer o pedido certo.

APIs no Seu Dia a Dia

Você usa dezenas de APIs por dia sem perceber.



Entrar com Google

O site pergunta à API do Google quem é você.



Pagamento por PIX

O app do banco envia o pagamento por API.



Busca de CEP

A API (ex.: ViaCEP) devolve o endereço em JSON.



Previsão do tempo

O app consulta uma API de clima.



O front-end conversa com o Laravel por API — é o que vamos construir!

Requisição e Resposta

Toda conversa com uma API é um par: **pedido** enviado → **resposta** recebida.

Requisição (Request)

Método	`GET, POST, PUT...`
URL	`/api/equipamentos`
Headers	`Accept: json`
Body	`{ dados em JSON }`

Resposta (Response)

Status	`200, 201, 404...`
Headers	`Content-Type: json`
Body	`{ dados em JSON }`

(No pedido, o body só vai em POST e PUT)

JSON — JavaScript Object Notation

Texto leve e fácil de ler: é o formato da grande maioria das APIs.

equipamento.json

```
{
  "id": 1,
  "nome": "Notebook Dell",
  "marca": "Dell",
  "numero_serie": "DL-2024-001",
  "status": "disponivel"
}
```

{ }
Objeto: abre e fecha os dados

"chave": valor
Cada informação é um par

" "
Chaves sempre com aspas duplas

,
Vírgula entre itens (não no último)

Tipos de Dados no JSON

Seis tipos — e só eles. Repare nas cores de cada valor.

```json
{
  "nome": "Projetor Epson",
  "quantidade": 3,
  "emprestado": false,
  "observacao": null,
  "acessorios": ["cabo HDMI", "controle"],
  "local": { "sala": "Lab 2", "bloco": "B" }
}
```

`string`	texto entre aspas
`number`	inteiro ou decimal
`boolean`	true ou false
`null`	sem valor
`array`	lista: [a, b]
`object`	objeto: { ... }

XML — eXtensible Markup Language

Formato com **tags**, parecido com HTML. Comum em sistemas mais antigos.

equipamento.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<equipamento>
  <id>1</id>
  <nome>Notebook Dell</nome>
  <marca>Dell</marca>
  <numero_serie>DL-2024-001</numero_serie>
  <status>disponivel</status>
</equipamento>
```

🏷️ Toda tag aberta precisa ser fechada

🌳 Um único elemento raiz

🔤 Aceita atributos: <equipamento id="1">

📌 A **NF-e** (Nota Fiscal Eletrônica) é enviada em XML até hoje.

JSON x XML — Lado a Lado

A mesma informação nos dois formatos.

JSON

```json
{
  "nome": "Notebook Dell",
  "status": "disponivel"
}
```

XML

```xml
<equipamento>
  <nome>Notebook Dell</nome>
  <status>disponivel</status>
</equipamento>
```

	JSON	XML
Tamanho	Menor, mais leve	Maior (abre e fecha tags)
Leitura	Simples e direta	Mais verbosa
Onde aparece	APIs REST, apps, Laravel	NF-e, SOAP, sistemas legados

TEMA 2 — ATIVIDADE RÁPIDA

Encontre os Erros 🔍

Este JSON é **inválido**. Quantos erros você encontra?

mouse.json

```
{
  nome: "Mouse Logitech",
  "marca": 'Logitech',
  "status": "disponivel",
}
```

3

erros escondidos

Discuta com o colega do lado

🌐 Dica: o site **jsonlint.com** valida qualquer JSON e aponta a linha do erro.

Correção

Os três erros mais comuns de quem está começando com JSON.

mouse.json ✓

```
{
  "nome": "Mouse Logitech",
  "marca": "Logitech",
  "status": "disponivel"
}
```

1 Chave sem aspas

`nome` → `"nome"`

2 Aspas simples

`'Logitech'` → `"Logitech"`

3 Vírgula no último item

`"disponivel",` → `"disponivel"`

Métodos HTTP e o CRUD

Cada operação do CRUD tem um **método HTTP** correspondente.

C	**Create**	POST	Cadastra um novo equipamento
R	**Read**	GET	Busca (lista todos ou um só)
U	**Update**	PUT	Altera um equipamento (ou PATCH)
D	**Delete**	DELETE	Remove um equipamento

Cores parecidas com as do Insomnia

Rotas REST da API de Equipamentos

A URL é sempre o **recurso no plural** — o que muda é o método.

Método	URL	Controller	Ação
GET	/api/equipamentos	index	Lista todos
POST	/api/equipamentos	store	Cadastra
GET	/api/equipamentos/{id}	show	Mostra um
PUT	/api/equipamentos/{id}	update	Atualiza
DELETE	/api/equipamentos/{id}	destroy	Remove

Códigos de Status HTTP

O número que vem na resposta diz se deu certo — e, se não deu, de quem é a culpa.

200	**OK** Deu certo (GET, PUT)	**201**	**Created** Registro criado (POST)	**204**	**No Content** Removido, sem corpo (DELETE)
404	**Not Found** Rota ou registro não existe	**422**	**Unprocessable** Dados inválidos (validação)	**500**	**Server Error** Erro no código do servidor

🚦 **2xx** = sucesso • **4xx** = erro de quem pediu • **5xx** = erro do servidor

Ponto de Partida

O projeto Laravel 12 **já existe**. Vamos expor o CRUD de Equipamentos como API.

1 **Habilitar API** — `install:api`

›

2 **Migration e Model** — `$fillable`

›

3 **Controller API** — `5 métodos`

›

4 **Rotas** — `apiResource`

›

5 **Testar** — `Insomnia`

Campos do equipamento: `nome` • `marca` • `numero_serie` • `status`

⚠️ Migration e model já existem? Então só **confira os campos** — não precisa recriar.

Passo 1 — Habilitar as Rotas de API

No Laravel 11/12 o arquivo **routes/api.php** não vem pronto. Um comando resolve:

Terminal

```
$ php artisan install:api
```

📄 Cria o arquivo **routes/api.php**

🔑 Instala o **Laravel Sanctum**

🧭 Todas as rotas ganham **o prefixo /api**

💬 Se o terminal perguntar se deseja rodar as migrations, responda **yes**.

Passo 2 — Migration de Equipamentos

A tabela no banco. Confira se a sua tem estes campos.

database/migrations/xxxx_create_equipamentos_table.php

```php
Schema::create('equipamentos', function (Blueprint $table) {
    $table->id();
    $table->string('nome', 100);
    $table->string('marca', 50);
    $table->string('numero_serie', 30);
    $table->string('status')->default('disponivel');
    $table->timestamps();
});
```

```
$ php artisan make:model Equipamento -m
```

```
$ php artisan migrate
```

Passo 2 — Model Equipamento

O model representa a tabela e libera os campos para gravação.

app/Models/Equipamento.php

```php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Equipamento extends Model
{
    protected $fillable = [
        'nome', 'marca',
        'numero_serie', 'status',
    ];
}
```

✅

$fillable

Campos liberados para `create()` e `update()`.

⚠️ Esqueceu?

Erro **500**: MassAssignmentException

Passo 3 — Criar o Controller da API

O Artisan gera o controller com os 5 métodos prontos para preencher.

Terminal

```
$ php artisan make:controller Api/EquipamentoController --api --model=Equipamento
```

Api/

Cria dentro da pasta Controllers/Api (organização)

--api

Só os métodos de API: sem create e edit (telas)

--model

Já recebe o Equipamento nos métodos

Métodos gerados: `index` `store` `show` `update` `destroy`

Controller — index (Listar)

Retorna todos os equipamentos. O Laravel converte para JSON sozinho.

EquipamentoController.php

```php
public function index()
{
    return Equipamento::all();
}
```

🔁 Web: `view(...)` → API: **os dados**

Resposta • 200 OK

```json
[
  {
    "id": 1,
    "nome": "Notebook Dell",
    "status": "disponivel"
  },
  { ... }
]
```

Controller — store (Cadastrar)

EquipamentoController.php

```php
public function store(Request $request)
{
    $dados = $request->validate([
        'nome' => 'required|string|max:100',
        'marca' => 'required|string|max:50',
        'numero_serie' => 'required|string|max:30',
        'status' => 'nullable|string|max:20',
    ]);
    $equipamento = Equipamento::create($dados);
    return response()->json($equipamento, 201);
}
```

1 validate()

Dados inválidos? Para aqui e devolve **422**.

2 create()

Grava no banco os campos do $fillable.

3 201

Status **Created**: registro criado.

Controller — show e destroy

EquipamentoController.php

```php
public function show(Equipamento $equipamento)
{
    return $equipamento;
}

public function destroy(Equipamento $equipamento)
{
    $equipamento->delete();
    return response()->noContent();
}
```



Route Model Binding

Busca pelo **{id}** da URL. Não achou? **404**.

noContent()

Status **204**, sem corpo.

Controller — update (Atualizar)

EquipamentoController.php

```php
public function update(Request $request, Equipamento $equipamento)
{
    $dados = $request->validate([
        'nome' => 'required|string|max:100',
        'marca' => 'required|string|max:50',
        'numero_serie' => 'required|string|max:30',
        'status' => 'nullable|string|max:20',
    ]);
    $equipamento->update($dados);
    return $equipamento;
}
```

Mesma validação do store + **update()**. Retorna o equipamento atualizado (**200**).

Passo 4 — Registrar as Rotas

Uma única linha cria as 5 rotas da API.

routes/api.php

```php
<?php
use App\Http\Controllers\Api\EquipamentoController;
use Illuminate\Support\Facades\Route;

Route::apiResource('equipamentos', EquipamentoController::class);
```

```
$ php artisan route:list --path=api
```

📋 Devem aparecer **5 rotas**.

⚠️ **No routes/api.php — não no web.php!**

Subindo o Servidor

Com o servidor rodando, a API já responde no endereço abaixo.

Terminal

```
$ php artisan serve
   INFO  Server running on [http://127.0.0.1:8000].
```

🌐 **http://127.0.0.1:8000/api/equipamentos**

🖥️ Deixe esse terminal **aberto**. Fechou? A API para de responder.

O que é o Insomnia?

Programa gratuito para enviar requisições e ver as respostas da API.

Monta a requisição

Método, URL, headers e body em JSON.

Mostra a resposta

Status, tempo e o JSON formatado.

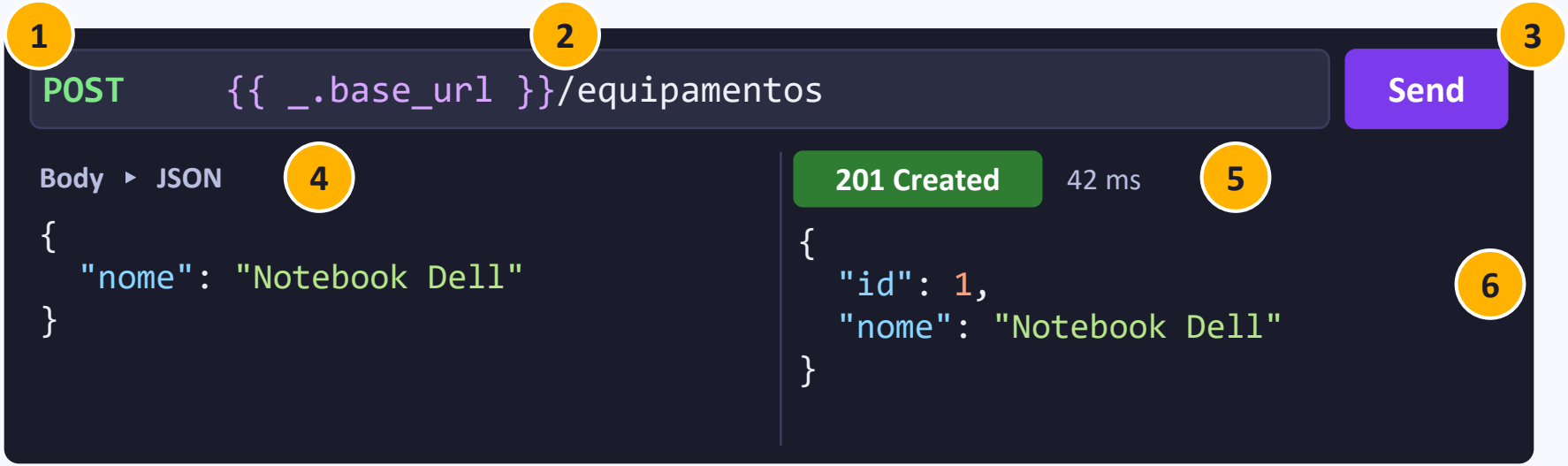
Organiza

Coleções e variáveis de ambiente.

💡 O navegador só faz **GET** pela barra de endereço. Para **POST, PUT e DELETE** precisamos do Insomnia.

Download: insomnia.rest

Conhecendo a Tela



1 **Método HTTP**

2 **URL do endpoint**

3 **Send (Ctrl+Enter)**

4 **Body / Headers**

5 **Status e tempo**

6 **Resposta (JSON)**

Coleção e Ambiente

Organize uma vez, reaproveite em todas as requisições.

1. **Criar a coleção**
   Create → Request Collection → “API Equipamentos”

2. **Criar o ambiente**
   Manage Environments → Base Environment

3. **Usar na URL**
   Digite base_url e escolha a variável

Base Environment

```
{
  "base_url": "http://127.0.0.1:8000/api"
}
```

URL na requisição

```
{{ _.base_url }}/equipamentos
```

Mudou o servidor? Troca só o ambiente.

Header Obrigatório: Accept

Avisa ao Laravel: “quero a resposta em **JSON**”.

```
Accept: application/json
```

❌ Sem o header

Erro de validação vira **redirect (302)** ou uma **página HTML** de erro. Difícil de entender no Insomnia.

✅ Com o header

Erros chegam em **JSON** com status **422** e a lista de campos inválidos.

Teste 1 — POST: Cadastrar

Body ▸ **JSON**, digite os dados e Send. Cadastre **pelo menos 2**!

POST {{ _.base_url }}/equipamentos — **Send**

Body ▸ JSON

```
{
  "nome": "Notebook Dell",
  "marca": "Dell",
  "numero_serie": "DL-2024-001",
  "status": "disponivel"
}
```

201 Created 58 ms

```
{
  "nome": "Notebook Dell",
  "marca": "Dell",
  "numero_serie": "DL-2024-001",
  "status": "disponivel",
  "id": 1
}
```

Teste 2 — GET: Listar e Buscar

GET não envia body. Só método + URL + Send.

GET {{ _.base_url }}/equipamentos — Send

Body

(sem body — GET só busca dados)

200 OK 21 ms

```
[
  { "id": 1, "nome": "Notebook Dell", ... },
  { "id": 2, "nome": "Projetor Epson", ... }
]
```

GET	/equipamentos/1	→ 200 OK — um equipamento
GET	/equipamentos/99	→ 404 Not Found — não existe

Teste 3 — PUT: Atualizar

O **id** vai na URL; os dados novos (status **em_uso**) vão no body.

PUT {{ _.base_url }}/equipamentos/1 — Send

Body ▸ JSON

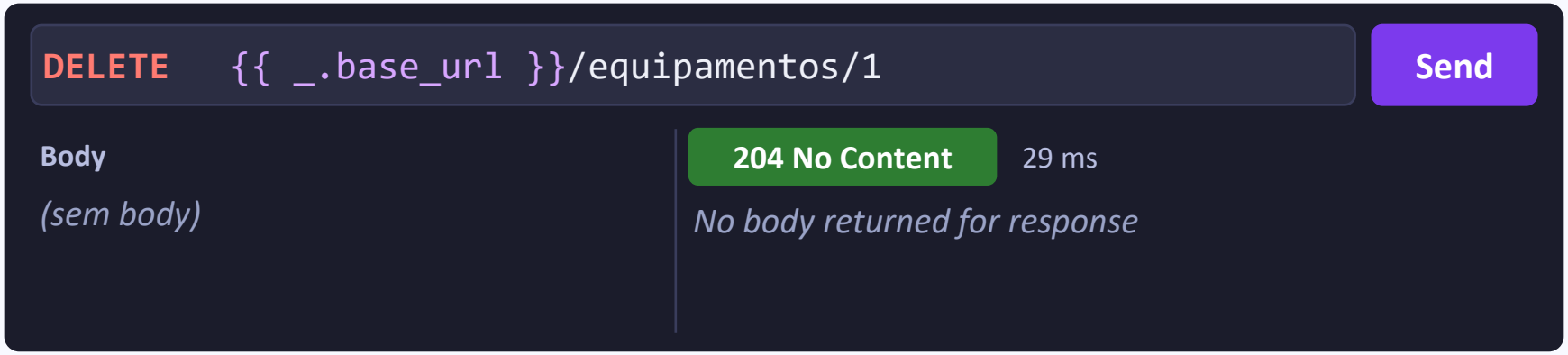
```
{
  "nome": "Notebook Dell",
  "marca": "Dell",
  "numero_serie": "DL-2024-001",
  "status": "em_uso"
}
```

200 OK 37 ms

```
{
  "id": 1,
  "nome": "Notebook Dell",
  "marca": "Dell",
  "numero_serie": "DL-2024-001",
  "status": "em_uso"
}
```

Teste 4 — DELETE: Remover

Resposta **204**: deu certo, mas não há nada para mostrar.



Para confirmar:

GET `/equipamentos/1` → **404 Not Found (foi removido ✓)**

Teste 5 — Validação (Erro 422)

Mande dados incompletos de propósito e leia a resposta.

POST {{ _.base_url }}/equipamentos — **Send**

Body ▸ JSON

```
{
  "marca": "HP"
}
```

422 Unprocessable Content 33 ms

```
{
  "message": "The nome field is ...",
  "errors": {
    "nome": ["The nome field is required."],
    "numero_serie": ["The numero serie ..."]
  }
}
```

SOLUÇÃO DE PROBLEMAS

Erros Comuns e Como Resolver

Sintoma	Causa provável / solução
`404 em /api/equipamentos`	Faltou install:api ou a URL está sem /api
`Resposta em HTML ou 302`	Falta o header Accept: application/json
`500 MassAssignmentException`	Campo fora do $fillable no model
`422 com tudo preenchido`	Body não está em JSON no Insomnia
`419 Page Expired`	Rota criada no web.php em vez do api.php
`Couldn't connect to server`	O php artisan serve não está rodando

ATIVIDADE

Hora de Praticar

Em duplas, no projeto de vocês:

1. Faça os **5 testes** no Insomnia e tire print de cada resposta.
2. Adicione o campo **localizacao** (migration, $fillable, validação) e teste.
3. Provoque um **422** e um **404** de propósito e explique cada um.
4. **Desafio:** rota GET que lista só os equipamentos **disponíveis**.

✅ **Entrega:** coleção exportada do Insomnia (.json) + prints dos testes.